# Part 3 Cross-Language Comparison

23CSE455 Design Patterns, Case Study Report: Part 3 and its LLM records

**Team:** A13

**Case study:** Design Pattern Analysis in a Vue.js-Based Social Media Application

**Application:** Misskey (`https://github.com/misskey-dev/misskey`), baseline commit `53ac6808ae4921305e761c545863ee6593edd1f4`. Frontend: Vue 3.5.42; backend: NestJS 11.2.3 (package.json files in the checkout)..

**Date:** 2026-10-11

**Students:** AM.SC.U4CSE23156 Sreehari M; AM.SC.U4CSE23148 Rohit Prasanth; AM.SC.U4CSE23157 Sreyas S Sankar; AM.SC.U4CSE23173 Sreevisakh A; AM.SC.U4CSE23168 Vighnesh H.

**Source of this document:** Prepared by an AI assistant (Claude Sonnet 5.5) from the data files in this folder.

## 4 Part 3 Cross-language comparison

### 4.1 Selected instances

One representative verified instance per pattern type (15 types). The source check is the assessment of the cited Misskey code re-read at the baseline commit: 11 correct, 4 partial. The partial ones are kept as confirmed with the stated caveat.

| # | Pattern type | Misskey instance | Layer | Source check | Caveat |
|---|---|---|---|---|---|
| 1 | Strategy | Logging Output Strategy | backend (NestJS) | correct | |
| 2 | Factory | Logging Backend Factory | backend (NestJS) | partial | a simple factory function with one caller |
| 3 | Adapter | Telemetry Backend Adapter Layer | backend (NestJS) | correct | |
| 4 | Template Method | Misskey Chart Processing Framework | backend (NestJS) | correct | |
| 5 | Proxy | AnalyticsProxy | frontend (Vue) | correct | |
| 6 | Observer | PreferencesManager Observer | frontend (Vue) | correct | |
| 7 | Two-Level Cache | RedisSingleCache L1/L2 Cache | backend (NestJS) | correct | |
| 8 | Cache-Aside / Lazy Loading | Federated Instance Cache-Aside | backend (NestJS) | correct | |
| 9 | Publish-Subscribe | Global Redis Event Publication and Streaming Subscription | backend (NestJS) | correct | |
| 10 | Asynchronous Message Queue / Job Queue | Federation Delivery Asynchronous Job Queue | backend (NestJS) | correct | |
| 11 | Front Controller | Misskey API Front Controller | backend (NestJS) | partial | shared pre-processing, but one route per endpoint |
| 12 | Router | Nirax Frontend Router | frontend (Vue) | correct | |
| 13 | Service Layer | Misskey Core Application Service Layer | backend (NestJS) | partial | one service (AnnouncementService) read in detail |
| 14 | Plugin Architecture / Microkernel | Misskey Frontend Plugin Architecture | frontend (Vue) | correct | |
| 15 | Repository | Misskey Persistence Repository Layer | backend (NestJS) | partial | the repository abstraction is TypeORM's; Misskey adds insertOne and DI wiring |

Three further audit candidates (Mediator, Object Pool, Streaming Connection Factory) were uncertain and are not implemented.

### 4.2 Common specifications

Each pattern has one fixture in `Part3/test-cases/<pattern>.txt`: one case per line with an input and an expected result string, including error cases. All four runners read the same fixture path from the command line; no test embeds expected values. The 15 fixtures hold 97 cases, so 388 executions (97 x 4 languages). Behaviour of

Misskey's frameworks (Redis, TypeORM, BullMQ, Vue, Fastify) is replaced by small in-memory models; each README lists what is not modelled and which rules the model added.

### 4.3 Implementations

Layout: `Part3/code/pattern_<name>/{java,python,javascript,cpp}/`, each with one production file, one separate test runner and a README with the run command. Versions: OpenJDK 17.0.19 (javac 17.0.19); CPython 3.14.6; Node.js v24.15.0; g++ 16.1.1 20260625, -std=c++17. Example (Factory, Python): `python3 -B test_logging_factory.py ../../../test-cases/factory.txt`, run from `Part3/code/pattern_factory/python/`. The exact command of every snippet is in the `test_command` column of `data/cross_language.csv`.

### 4.4 Actual results

All 60 implementations compiled and ran; every test runner printed `passed N/N` (388 of 388 case executions) and exited 0. Logs: `Part3/evidence/test-logs/<pattern>_<language>.log`. The 60 test commands were re-run from scratch copies on 2026-10-11 with the same result and no compiler warnings. No result is marked not_run.

| Pattern type | Java | Python | JavaScript | C++ |
|---|---|---|---|---|
| Strategy | 5/5 passed | 5/5 passed | 5/5 passed | 5/5 passed |
| Factory | 6/6 passed | 6/6 passed | 6/6 passed | 6/6 passed |
| Adapter | 8/8 passed | 8/8 passed | 8/8 passed | 8/8 passed |
| Template Method | 5/5 passed | 5/5 passed | 5/5 passed | 5/5 passed |
| Proxy | 5/5 passed | 5/5 passed | 5/5 passed | 5/5 passed |
| Observer | 8/8 passed | 8/8 passed | 8/8 passed | 8/8 passed |
| Two-Level Cache | 7/7 passed | 7/7 passed | 7/7 passed | 7/7 passed |
| Cache-Aside / Lazy Loading | 7/7 passed | 7/7 passed | 7/7 passed | 7/7 passed |
| Publish-Subscribe | 5/5 passed | 5/5 passed | 5/5 passed | 5/5 passed |
| Asynchronous Message Queue / Job Queue | 6/6 passed | 6/6 passed | 6/6 passed | 6/6 passed |
| Front Controller | 7/7 passed | 7/7 passed | 7/7 passed | 7/7 passed |
| Router | 7/7 passed | 7/7 passed | 7/7 passed | 7/7 passed |
| Service Layer | 8/8 passed | 8/8 passed | 8/8 passed | 8/8 passed |
| Plugin Architecture / Microkernel | 7/7 passed | 7/7 passed | 7/7 passed | 7/7 passed |
| Repository | 6/6 passed | 6/6 passed | 6/6 passed | 6/6 passed |

### 4.5 Static and dynamic comparison

Source LOC: lizard NLOC of the production file (no blanks, comments or tests) [6]. Maximum function cyclomatic complexity (CC): largest function CCN from lizard 1.24.1; radon agrees on 14 of 15 Python files. The one difference, Router (radon 11, lizard 10), is explained by radon counting the else clause of a loop, which lizard does not: the installed radon 6.0.1 source adds one for a for/while with orelse [5]. Time: wall-clock of one process that starts, reads the fixture, runs every case and exits, compiled beforehand; one warm-up run, then five timed runs on one machine; the median of the five is reported (raw runs: `Part3/evidence/timing/raw_timing.csv`). The warm-up run is executed by the harness but its time was not stored. Start-up is included, so times mostly compare runtime start-up. Machine: Linux-7.1.2-zen3-1-zen-x86_64-with-glibc2.43, x86_64, Intel(R) Core(TM) i7-9750H CPU @ 2.60GHz.

Each cell: source LOC / max function CC / median ms.

| Pattern type | Java | Python | JavaScript | C++ |
|---|---|---|---|---|
| Strategy | 31 / 2 / 80.2 | 21 / 2 / 33.1 | 18 / 2 / 45.8 | 43 / 2 / 3.3 |
| Factory | 23 / 4 / 83.1 | 14 / 4 / 23.9 | 14 / 4 / 47.3 | 33 / 4 / 3.4 |
| Adapter | 52 / 2 / 89.1 | 36 / 2 / 24.5 | 26 / 2 / 45.4 | 55 / 2 / 3.7 |
| Template Method | 37 / 4 / 82.3 | 27 / 4 / 23.8 | 25 / 4 / 47.8 | 36 / 4 / 3.3 |
| Proxy | 28 / 2 / 77.1 | 29 / 2 / 24.7 | 17 / 3 / 45.6 | 28 / 2 / 3.6 |
| Observer | 22 / 3 / 93.3 | 18 / 4 / 27.3 | 13 / 3 / 46.1 | 35 / 4 / 3.4 |
| Two-Level Cache | 68 / 3 / 89.0 | 47 / 3 / 24.2 | 56 / 3 / 50.0 | 67 / 3 / 3.3 |
| Cache-Aside / Lazy Loading | 49 / 3 / 87.4 | 47 / 3 / 27.6 | 47 / 3 / 43.5 | 67 / 3 / 3.3 |
| Publish-Subscribe | 35 / 2 / 96.0 | 25 / 2 / 36.8 | 26 / 4 / 46.3 | 43 / 2 / 3.4 |

| Pattern type | Java | Python | JavaScript | C++ |
|---|---|---|---|---|
| Asynchronous Message Queue / Job Queue | 65 / 6 / 92.1 | 42 / 6 / 27.8 | 49 / 7 / 46.2 | 83 / 5 / 3.4 |
| Front Controller | 30 / 6 / 94.4 | 24 / 6 / 27.3 | 21 / 9 / 49.4 | 35 / 5 / 3.3 |
| Router | 57 / 9 / 91.9 | 38 / 10 / 27.3 | 36 / 6 / 45.2 | 69 / 9 / 3.4 |
| Service Layer | 78 / 6 / 104.2 | 61 / 5 / 55.3 | 54 / 5 / 46.5 | 92 / 6 / 3.5 |
| Plugin Architecture / Microkernel | 27 / 3 / 79.0 | 16 / 4 / 25.0 | 19 / 2 / 46.3 | 35 / 3 / 3.7 |
| Repository | 52 / 3 / 92.3 | 40 / 4 / 23.9 | 40 / 3 / 51.0 | 58 / 6 / 3.5 |

| Language | Median LOC (range) | Max function CC | Median process time (range) |
|---|---|---|---|
| Java | 37 (22-78) | 9 | 77.1-104.2 ms |
| Python | 29 (14-61) | 10 | 23.8-55.3 ms |
| JavaScript | 26 (13-56) | 9 | 43.5-51.0 ms |
| C++ | 43 (28-92) | 9 | 3.3-3.7 ms |

## 4.6 Comparison

Observations come from the `language_notes` column and the per-language READMEs. Java and C++ need more lines because of declarations, closing braces and includes (C++ NLOC counts `#include` lines). Example, Strategy: Java: The strategy is a nominal interface (`LogBackend`); the compiler rejects a backend without `write`, and null is rejected by an explicit check in `setBackend`. Python: `LogBackend` is a `typing.Protocol` used as documentation; backends do not inherit from it and nothing is enforced at runtime, so a backend without `write` only fails when called.

Example, Observer: JavaScript: The subject extends Node's built-in `EventEmitter`, so `on`/`off`/`emit` come from the runtime (the TypeScript source uses EventEmitter3 the same way). `off` with an undefined listener throws a TypeError, so the test passes a never-registered function for the unknown-listener case. C++: `std::function` objects cannot be compared, so `on` returns an integer handle that `off` takes; notification runs over a copy of the listener list.

## 4.7 Technology discussion

**Interfaces and structural typing.** Java declares an interface that classes explicitly implement [4]; a class that does not implement the required method is rejected at compile time, as the Java Strategy snippet relies on. Python offers the same role through `typing.Protocol`, which is structural (static duck typing) and mainly used by static type checkers; it is checked at runtime only for `@runtime_checkable` protocols and then only for attribute presence [1]. The Python Strategy snippet therefore uses the protocol as documentation, and a backend without `write` fails only when called (see the Strategy `language_notes` above).

**Events.** Node.js provides `EventEmitter` as a built-in class that calls all listeners synchronously in the order in which they were registered [2]. The JavaScript Observer snippet uses it directly, while Misskey's own `frontend/src/preferences/manager.ts` imports the `eventemitter3` library, so the snippet reproduces the notification order and not the dependency. C++ has no such built-in; the snippet stores `std::function` objects, and because `std::function` defines equality only against `nullptr` (and that operator was removed in C++20) [3], listeners are removed through an integer handle returned by on.

**Measurement tools.** lizard reports NLOC and cyclomatic complexity for Java, Python, JavaScript and C++ [6] and, as its authors state, measures how complex code looks rather than how complex it is. Radon's rule table adds one point per decision such as for, while, boolean operators and comprehensions [5], and its implementation also counts a loop else clause. In this study lizard dropped JavaScript methods named get, set or delete, so the harness measures a temporary copy with those names suffixed (the production files are unchanged).

**What the frameworks hide.** Misskey gets queueing (BullMQ), caching (Redis), persistence (TypeORM) and routing (Fastify) from dependencies; the snippets re-implement only the collaboration, so the comparison shows how each language expresses the pattern structure, not how Misskey performs.

## 4.8 Scope note

The assigned topic is Design Pattern Analysis in a Vue.js-Based Social Media Application. Misskey is a full-stack application: a Vue 3 frontend and a NestJS backend. 4 of the 15 selected instances are in the frontend (Proxy, Observer, Router, Plugin Architecture) and 11 are in the backend. The pattern selection follows the confirmed instances of the audit, not the frontend alone, so a reader who expects only Vue.js code may see this as a scope concern.

# 5 LLM records and human verification

The evaluations below were made by AI-assisted checks (re-reading cited source, re-running tests, recomputing medians, re-measuring LOC and CC, comparing the PDFs with the CSV by script); they were produced in the same model family that produced the artifacts, so they are not independent human verification. Review entries and evidence: `Part3/llm/verification/human_review_checklist.md`.

| Row | Task | Model and date | Prompt file | Output | Evaluation and correction | Verification / human review |
|---|---|---|---|---|---|---|
| 1 | Scope selection and source verification | ChatGPT GPT-6, 2026-10-11 | Part1/Part1_prompt.txt | Part1/patterns.csv; Part1/pattern_counts.csv | LLM-assisted candidate discovery and classification. 22 pattern collaborations were retained after source-oriented checks and a final missed-pattern scan. Exact line ranges and representative snippets should be verified against the frozen source before final submission. | reviewer Sreehari M, 11/10/2026: corrected |
| 2 | Writing the 60 snippets, tests, fixtures, READMEs | Antigravity Agent (Google DeepMind) 2026-10-11 | Part2/llm/inputs/Change1_prompt.txt | Part2/llm/outputs/Change1_output.txt | Change 1 implementation fulfilled all acceptance criteria; 21/21 focused tests passed; committed cleanly as 608b45de20dce0a78459025320bfdeb0c4e47a78 | reviewer Sreehari M, 11/10/2026: corrected |
| 3 | Measurement harness and evidence | Antigravity Agent (Google DeepMind) 2026-10-11 | Part2/llm/inputs/Change2_prompt.txt | Part2/llm/outputs/Change2_output.txt | Change 2 implementation fulfilled all acceptance criteria; 21/21 focused tests passed; committed cleanly as 9a4398a103e392a0ea634eaf521470985985145d | reviewer Sreehari M, 11/10/2026: disputed |
| 4 | Review and corrections | Antigravity Agent (Google DeepMind) 2026-10-11 | Part2/llm/inputs/Change3_prompt.txt | Part2/llm/outputs/Change3_output.txt | Change 3 implementation fulfilled all acceptance criteria; 29/29 focused tests passed; committed cleanly as 39e3c987488d59045cc3ed5cfe532f6f32c0936d | reviewer Sreehari M, 11/10/2026: confirmed |
| 5 | Verification session of 2026-10-11 | Antigravity Agent (Google DeepMind) 2026-10-11 | Part2/llm/inputs/Change4_prompt.txt | Part2/llm/outputs/Change4_output.txt | Change 4 implementation fulfilled all acceptance criteria; 20/20 focused tests passed; committed cleanly as b3040278d3e63cf4ced915f9623b95969ff42010 | reviewer Sreehari M, 11/10/2026: disputed |
| 6 | Slide deck (A13_slide.pdf) | Claude Sonnet 5.5 (claude-sonnet-5-5); 2026-10-10 | Part3/llm/part3_prompt.txt | Part3/llm/outputs/selection_phase_output.md | partial: Counts, commit and cited locations are supported, but the original confidence classifications were too strong for Factory, Front Controller, Service Layer and Repository; the three uncertain candidates remain excluded. The audit-results/validation.csv file was unavailable in the submitted repository. Human review outcome: corrected. | reviewer Sreehari M, 11/10/2026: corrected |
| 7 | This report (A13_Part3_Report.pdf) | Claude Sonnet 5.5 (claude-sonnet-5-5); 2026-10-10 | Part3/llm/part3_prompt.txt | Part3/llm/outputs/implementation_phase_output.md | correct: All 60 implementations pass the recorded shared-fixture tests across four languages. The Observer behaviour and README instructions were corrected; equivalence is limited to the tested cases. Human review outcome: corrected. | reviewer Sreehari M, 11/10/2026: corrected |

**Review status of the 7 records:** review entries recorded by Sreehari M (AM.SC.U4CSE23156), dated 11/10/2026 and completed with ChatGPT, in section 10 of `Part3/llm/verification/human_review_checklist.md`: confirmed 1, corrected 4, disputed 2. The AI-assisted verification (executed checks, source reading, script comparison) preceded the review; row 5 was produced and checked by the same model family.

`part3_prompt.txt` holds prompts 1-9 of the first session; the prompts for rows 6 and 7 are D3 and D5 in `deliverables_session_prompts.txt`.

Generated code: the 60 snippets, tests, fixtures and READMEs were produced by the model in row 2 and corrected after review (the Observer snippets were changed to match the source; seven corrections in `Part3/llm/verification/implementation_verification.md`). They were used as submitted after the tests in section 4.4 passed.

## 6 Findings and limitations

• All 60 equivalent implementations pass their shared fixtures; 11 of 15 instances match the cited source fully and 4 only partly.

• The warm-up run is executed by the measurement script but its time was not stored, so it cannot be shown from the output files.

• Timing is from one laptop without CPU pinning and includes process start-up and fixture reading.

• Cyclomatic complexity for Java, JavaScript and C++ comes from one tool (lizard); radon differs on one Python file (Router, 11 vs 10).

• The Observer oldValue behaviour (the emitted oldValue equals the new value) was reproduced by running the verbatim Misskey lines in a stub class, not inside Misskey's frontend.

• The models are small in-memory simplifications; equality of results is shown for the 97 fixture cases only.

• 11 of the 15 instances are backend code; see the scope note (section 4.8).

• The first-version generated code is not stored as files; only its corrected final form is in this folder.

## References

[1] Python Software Foundation, "typing - Support for type hints", Python 3 documentation, https://docs.python.org/3/library/typing.html (consulted 2026-10-11).
[2] OpenJS Foundation, "Events", Node.js documentation, https://nodejs.org/api/events.html (consulted 2026-10-11).
[3] cppreference.com, "std::function", https://en.cppreference.com/w/cpp/utility/functional/function (consulted 2026-10-11).
[4] Oracle, "The Java Language Specification, Java SE 17 Edition, Chapter 9: Interfaces", https://docs.oracle.com/javase/specs/jls/se17/html/jls-9.html (consulted 2026-10-11).
[5] Radon documentation, "Introduction to Code Metrics", https://radon.readthedocs.io/en/latest/intro.html, and the installed radon 6.0.1 source, radon/visitors.py (consulted 2026-10-11).
[6] T. Yin, "Lizard", README, https://github.com/terryyin/lizard (consulted 2026-10-11).
[7] Misskey source at commit 53ac6808ae4921305e761c545863ee6593edd1f4, https://github.com/misskey-dev/misskey (files cited in Part3/README.md).